

RAPPORT DE PROJET

Prédiction du Churn Client

dans le secteur des télécommunications

Analyse des données, modélisation prédictive
et recommandations pour la fidélisation des clients



AUTEUR
Samir NZAMBA



DATE
Septembre 2026



FORMATION
Mastère Architecte en Intelligence Artificielle
Fonderie de l'Image



Contexte

Ce projet s'inscrit dans le cadre du Mastère DPIA 1 à la Fonderie de l'Image. L'objectif est de construire un pipeline complet de Machine Learning pour prédire le churn des clients d'un opérateur télécom américain, c'est-à-dire leur risque de résilier leur abonnement dans les prochaines semaines.

Problématique

Acquérir un nouveau client coûte entre 5 et 7 fois plus cher que de fidéliser un client existant. Anticiper les résiliations avant qu'elles ne surviennent représente un enjeu financier majeur pour tout opérateur télécom.

Approche

Le dataset Telco Customer Churn (Kaggle, 7 043 clients, 21 variables) a été exploré, nettoyé et modélisé. Cinq algorithmes ont été comparés : Régression Logistique, Arbre de Décision, Random Forest, XGBoost et LightGBM.

Résultats

Le modèle retenu est XGBoost, avec un AUC-ROC de 0.8266 et un recall de 75.7 % sur le jeu de test. Sur 374 clients ayant effectivement churné, le modèle en détecte 283. L'analyse SHAP révèle que le type de contrat est le facteur le plus déterminant.

Livrables

Deux notebooks documentés, un pipeline modulaire src/, un dashboard Streamlit déployé en ligne, une API REST FastAPI, un rapport Evidently et un pipeline CI/CD GitHub Actions.

Métrique	Valeur	Interprétation
AUC-ROC	0.8266	Capacité discriminante solide (0.5 = aléatoire, 1.0 = parfait)
Accuracy	75.0 %	Proportion de prédictions correctes sur le jeu de test
Recall (Churné)	75.7 %	283 churners détectés sur 374 réels
F1-score	0.62	Équilibre Precision / Recall sur la classe minoritaire
Seuil de décision	0.10	Optimisé pour maximiser le recall

1) Introduction et contexte

Le churn désigne la proportion de clients qui résilient leur contrat sur une période donnée. Dans le secteur des télécommunications, la fidélisation est un enjeu central : les investissements pour attirer un nouveau client sont considérablement plus élevés que ceux requis pour maintenir un client existant.

Ce projet construit un modèle prédictif capable d'estimer, pour chaque client, la probabilité qu'il résilie son abonnement. L'approche retenue est celle du Machine Learning supervisé : on dispose de données historiques sur des clients ayant churné ou non, ce qui permet d'entraîner un modèle à reconnaître les profils à risque.

Objectifs du projet

Explorer et préparer le dataset de manière rigoureuse en documentant chaque décision. Comparer plusieurs algorithmes afin d'identifier le plus performant. Optimiser le modèle retenu par une recherche systématique des hyperparamètres. Déployer les résultats dans un dashboard interactif accessible à un public non technique. Mettre en place les éléments d'un projet data professionnel : tests unitaires, CI/CD, monitoring et API REST.

Périmètre

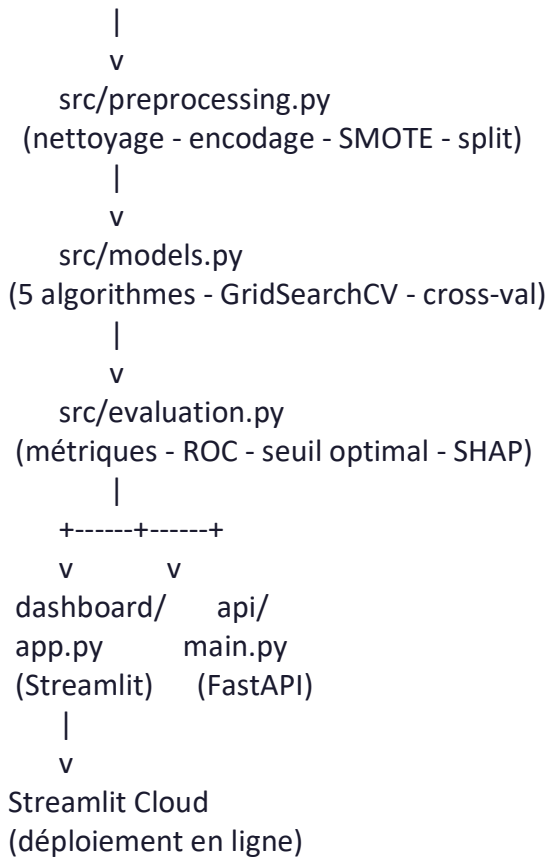
Dataset Telco Customer Churn (IBM / Kaggle) : 7 043 clients, 21 variables, variable cible Churn (Yes/No). Le projet ne traite pas les données en temps réel et ne s'applique pas à d'autres secteurs sans adaptation préalable.

2) Architecture et structure du projet

Vue d'ensemble

Le projet suit une architecture en pipeline linéaire : les données brutes sont chargées, nettoyées et encodées par le module de prétraitement, puis passées aux modules de modélisation et d'évaluation. Les résultats sont ensuite exposés via deux interfaces : un dashboard Streamlit pour les utilisateurs non techniques et une API REST FastAPI pour les intégrations externes.

Dataset Telco Customer Churn (Kaggle)



Structure du dépôt

Le dépôt est organisé de manière à séparer clairement les responsabilités entre les différents composants du projet.

```

churn_project/
+-- .github/
|  +-- workflows/
|     +-- ci.yml          # Pipeline CI/CD GitHub Actions
+-- api/
|  +-- main.py            # API REST FastAPI
+-- dashboard/
|  +-- app.py             # Dashboard Streamlit (5 pages)
+-- data/                 # Données brutes (non versionnées)
+-- exports/              # Visualisations, CSV, rapport Evidently
+-- models/               # Modèles sérialisés .pkl
+-- notebooks/
|  +-- 01_eda_preparation.ipynb # Exploration et préparation
|  +-- 02_modelisation_validation.ipynb # Modélisation et SHAP
  
```

```

+-- src/                # Modules Python réutilisables
| +-- preprocessing.py
| +-- models.py
| +-- evaluation.py
+-- tests/
| +-- test_preprocessing.py  # 6 tests unitaires
+-- requirements.txt
+-- README.md

```

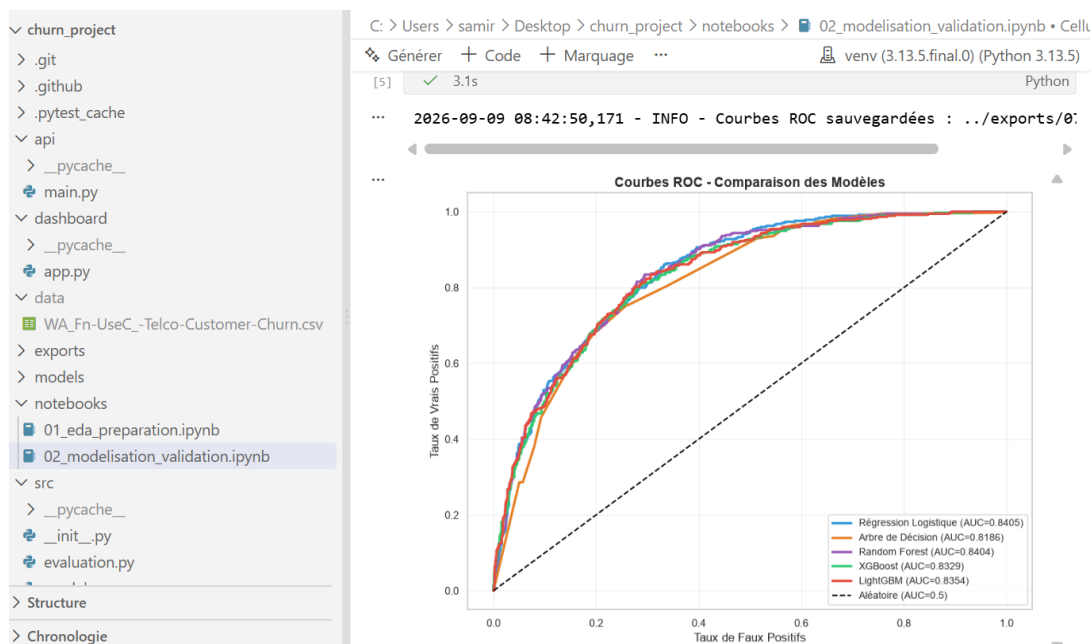


Figure 1. Structure du projet dans VS Code et exécution du notebook 02 avec les courbes ROC.

Modules src/

La logique métier est centralisée dans trois modules Python pour éviter toute duplication entre les notebooks, le dashboard et l'API.

Module	Fonctions principales	Utilisé par
preprocessing.py	load_raw_data(), clean_data(), encode_features(), split_and_scale(), run_preprocessing_pipeline()	Notebooks, Dashboard, API
models.py	get_models(), get_param_grids(), train_all_models(), optimize_model(), cross_validate_models()	Notebook 02
evaluation.py	evaluate_all_models(), plot_roc_curves(), plot_confusion_matrices(), plot_learning_curve(), find_optimal_threshold()	Notebook 02

Livrables produits

Livrable	Emplacement	Description
dataset_clean.csv	exports/	Dataset nettoyé et encodé
resultats_modeles.csv	exports/	Métriques comparatives des 8 modèles
shap_importance.csv	exports/	Importance SHAP par variable
evidently_report.html	exports/	Rapport de monitoring data drift
xgboost_best.pkl	models/	Modèle XGBoost optimisé sérialisé
shap_explainer.pkl	models/	Explainer SHAP pour les prédictions
optimal_threshold.pkl	models/	Seuil de décision optimisé (0.10)
app.py	dashboard/	Dashboard Streamlit 5 pages
main.py	api/	API REST FastAPI 4 endpoints

3) Données et préparation

Description du dataset

Le dataset Telco Customer Churn contient 7 043 clients et 21 colonnes. Les variables se répartissent en quatre catégories : informations démographiques, services souscrits, informations contractuelles et données de facturation. Le taux de churn est de 26.5 %, soit 1 869 clients sur 7 043.

Catégorie	Variables
Démographie	gender, SeniorCitizen, Partner, Dependents
Services	PhoneService, MultipleLines, InternetService, OnlineSecurity, OnlineBackup, DeviceProtection, TechSupport, StreamingTV, StreamingMovies
Contrat	Contract, PaperlessBilling, PaymentMethod
Facturation	tenure, MonthlyCharges, TotalCharges
Cible	Churn (Yes / No)

Nettoyage des données

L'exploration initiale a permis d'identifier quatre anomalies dans le dataset.

TotalCharges - type incorrect. Colonne stockée comme texte au lieu d'un nombre. Après conversion forcée en float, 11 valeurs vides sont apparues. Ces 11 clients ont tous une ancienneté de 0 mois : ils n'ont encore jamais été facturés. La valeur 0 a donc été attribuée, ce qui est la valeur logiquement correcte et non une imputation statistique arbitraire.

SeniorCitizen - encodage incohérent. Variable encodée 0/1 alors que toutes les autres variables binaires similaires utilisent Yes et No. Harmonisation appliquée : 0 devient No, 1 devient Yes.

customerID - variable sans valeur prédictive. L'identifiant client est unique par ligne et ne contient aucune information exploitable par un modèle. La colonne a été supprimée, ramenant le dataset de 21 à 20 variables.

Doublons - 22 lignes identiques. Volume non significatif (0.3 % du dataset). Conservées après analyse et anomalie documentée.

Analyse de la Variable Cible - Churn

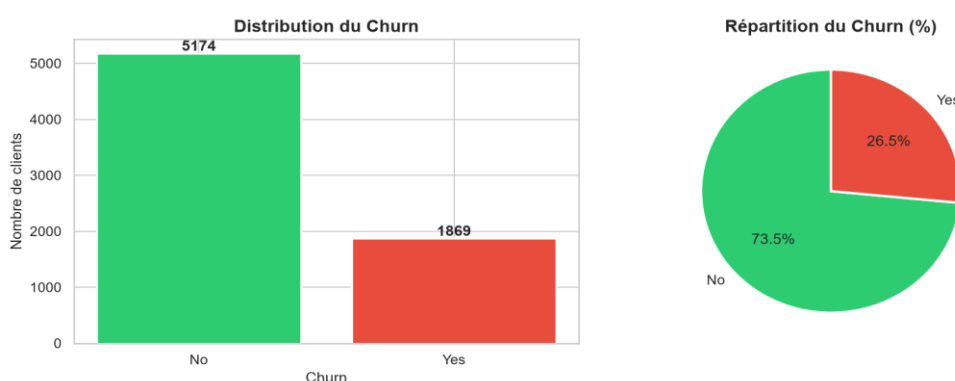


Figure 2. Distribution de la variable cible - 26.5% de churn sur 7 043 clients.

Encodage et normalisation

Les variables binaires (Yes/No) ont été converties via Label Encoding (Yes = 1, No = 0), soit 7 variables. Les variables à plusieurs modalités ont été traitées via One-Hot Encoding afin d'éviter

d'imposer un ordre artificiel entre les valeurs. Le dataset passe de 20 colonnes à 30 features exploitables.

Toutes les variables numériques ont été normalisées via un StandardScaler ajusté uniquement sur les données d'entraînement. Cette règle est fondamentale pour éviter le data leakage : si le scaler était ajusté sur l'ensemble des données, il introduirait une fuite d'information du test vers le train qui biaiserait l'évaluation.

Gestion du déséquilibre - SMOTE

Le dataset contient 73.5 % de non-churners contre 26.5 % de churners. Un modèle qui prédirait systématiquement "pas de churn" obtiendrait 73.5 % d'accuracy sans rien apprendre, ce qui illustre pourquoi l'accuracy seule est insuffisante sur ce type de problème.

SMOTE (Synthetic Minority Oversampling Technique) génère des exemples synthétiques plausibles de la classe minoritaire par interpolation entre des churners existants et leurs voisins proches. Résultat : 4 139 churners et 4 139 non-churners dans le train. SMOTE est appliqué uniquement après le découpage train/test, le test restant intact avec sa distribution naturelle.

Découpage train / test

80 % train (5 634 clients) / 20 % test (1 409 clients). Le paramètre stratify=y garantit 26.5 % de churn dans les deux ensembles. La reproductibilité est assurée par random_state=42 systématique dans l'ensemble du projet.

Difficultés rencontrées

La principale difficulté a concerné la cohérence des dimensions après SMOTE. Dans un premier temps, certains modèles étaient entraînés sur les données scalées post-SMOTE (8 278 lignes) mais évalués sur des données non scalées (5 634 lignes), ce qui générait une erreur de dimensions. La solution retenue a été de systématiquement utiliser X_train_scaled (post-SMOTE) pour tous les modèles, garantissant la cohérence des dimensions à chaque étape du pipeline.

4) Analyse exploratoire

L'analyse exploratoire a été conduite avant la modélisation dans le notebook 01, afin d'identifier les variables les plus liées au churn et de formuler des hypothèses métier qui guident ensuite la modélisation.

Variables démographiques

Le genre n'influence pas significativement le churn (26.9 % pour les femmes contre 26.2 % pour les hommes). Les seniors churnent à 41.7 % contre 23.6 % pour les non-seniors. Les clients sans partenaire (33.0 %) et sans personnes à charge (31.3 %) présentent des taux de churn plus élevés.

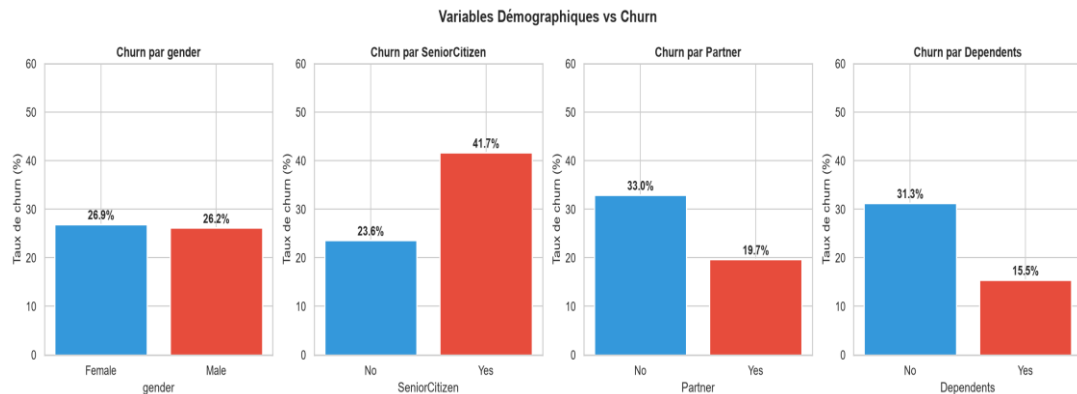


Figure 3. Taux de churn par variable démographique.

Type de contrat et ancienneté

Le type de contrat est la variable la plus discriminante identifiée lors de l'EDA : contrat mensuel 42.7 %, un an 11.3 %, deux ans 2.8 %. Les clients de moins de 12 mois churnent à 47.7 %, soit près d'un sur deux. Ce résultat oriente directement les recommandations métier formulées en section 9.

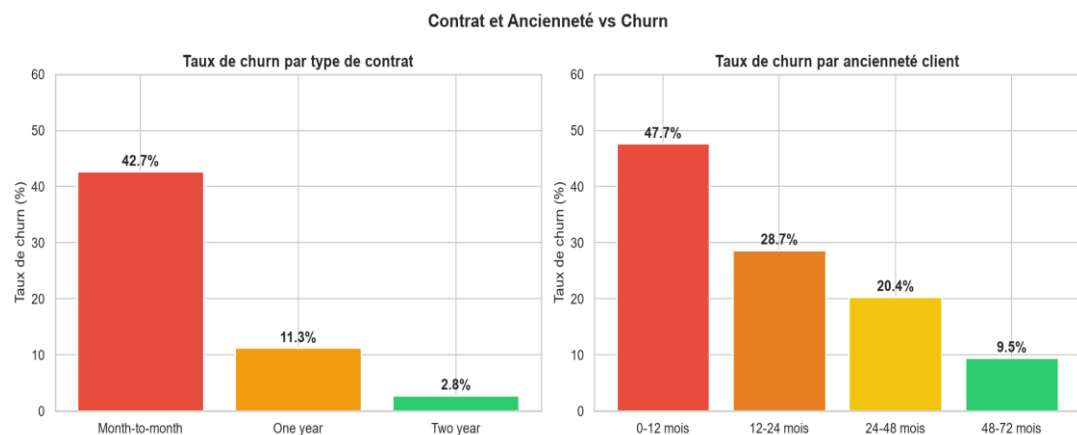


Figure 4. Taux de churn par type de contrat et ancienneté client.

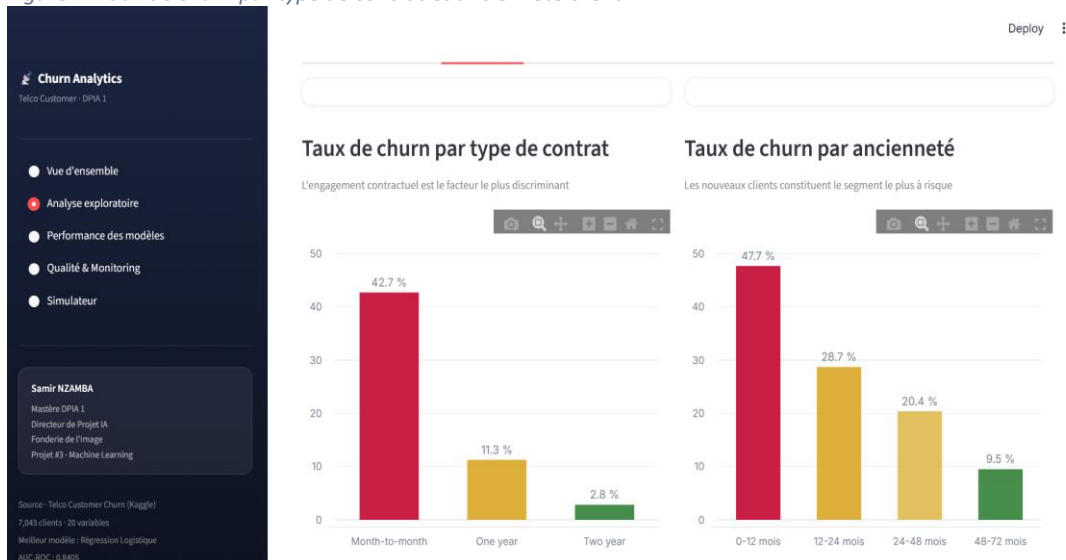


Figure 5. Visualisation interactive du dashboard - page Analyse exploratoire.

Services souscrits

Fiber optic présente un taux de churn paradoxal de 41.9 % pour un service premium. L'absence de sécurité en ligne (41.8 %) et de support technique (41.6 %) sont des facteurs de risque importants. En revanche, les clients sans internet du tout ne churne qu'à 7.4 %.

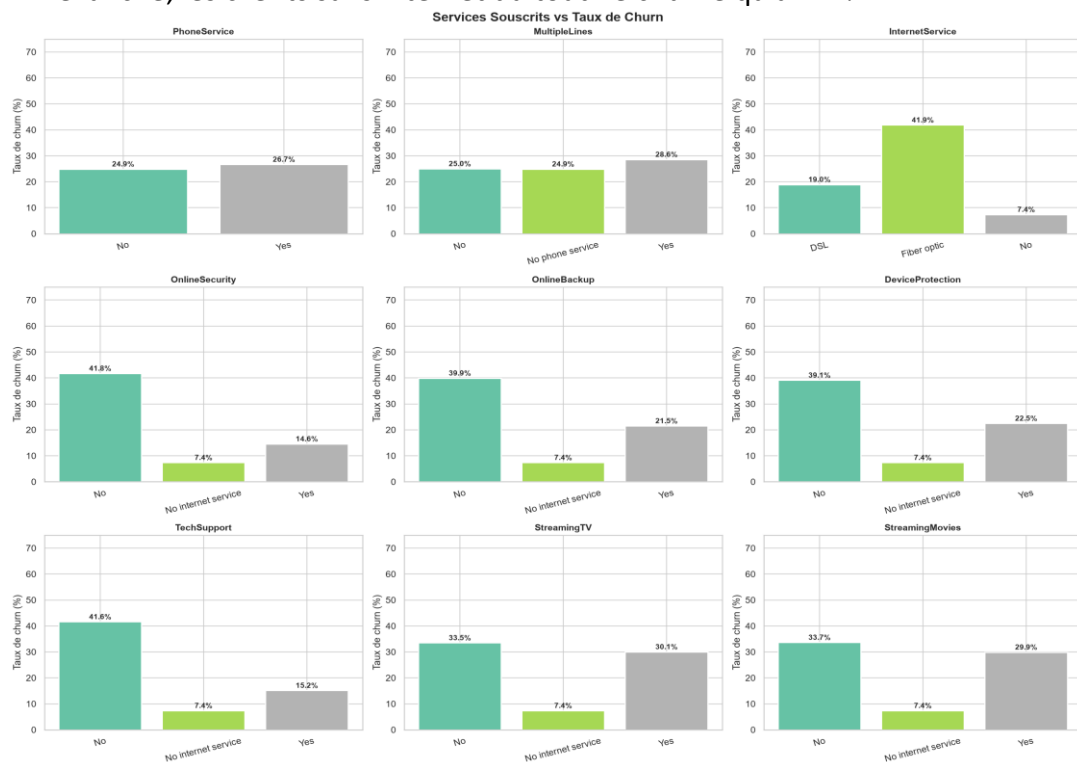


Figure 6. Taux de churn par service souscrit.

Variables numériques et corrélations

Forte corrélation entre tenure et TotalCharges (0.83), attendue puisque le total facturé augmente mécaniquement avec l'ancienneté. La mensualité est positivement corrélée au churn (+0.19). L'ancienneté est négativement corrélée (-0.35).

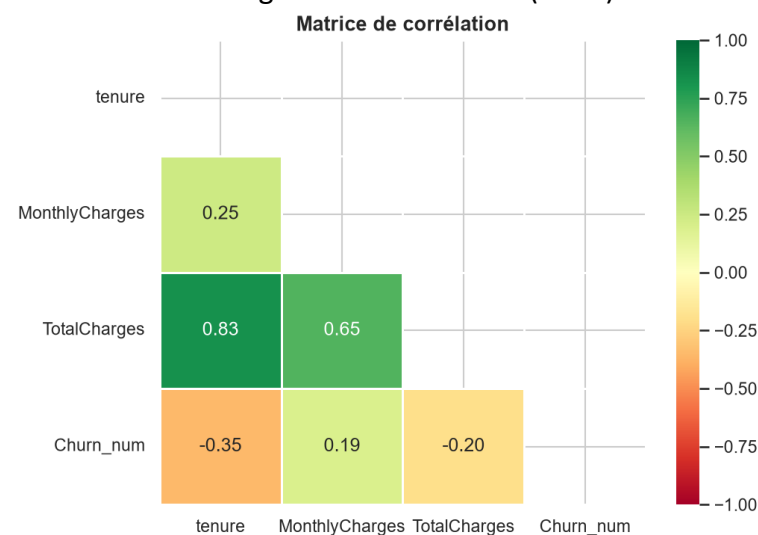


Figure 7. Matrice de corrélation des variables numériques.

5) Modélisation

Cinq algorithmes comparés

Régression Logistique. Modèle de référence (baseline). Calcule la probabilité de churn via une fonction sigmoïde. Simple, interprétable, rapide.

Arbre de Décision. Pose des questions successives sur les variables du client. Très lisible mais sensible au surapprentissage. Profondeur limitée à 5.

Random Forest. Vote majoritaire de 100 arbres entraînés sur des sous-échantillons aléatoires. Plus robuste qu'un seul arbre.

XGBoost. Gradient Boosting séquentiel : chaque arbre corrige les erreurs du précédent. Référence sur les données tabulaires dans les compétitions de Machine Learning. Modèle retenu.

LightGBM. Variante de XGBoost optimisée pour la vitesse. Résultats similaires sur ce dataset, validé comme alternative robuste.

Justification du choix de XGBoost

XGBoost a été retenu pour deux raisons. D'abord, sa feature importance native est plus précise car elle tient compte du gain réel apporté par chaque variable à chaque split, et non simplement de la fréquence d'utilisation. Ensuite, la disponibilité d'un explainer SHAP dédié aux modèles de gradient boosting permet une explicabilité individuelle plus fine.

6) Optimisation et évaluation

GridSearchCV

L'optimisation des hyperparamètres a été réalisée via GridSearchCV sur les trois modèles les plus prometteurs. Pour XGBoost, la grille couvre le nombre d'arbres (100 ou 200), la profondeur maximale (4, 6 ou 8), le taux d'apprentissage (0.01, 0.05 ou 0.1) et le sous-échantillonnage (0.8 ou 1.0). Cela représente 36 combinaisons, chacune évaluée sur 5 folds, soit 180 entraînements. La métrique de scoring utilisée est l'AUC-ROC, plus robuste que l'accuracy sur un dataset déséquilibré.

Validation croisée

La validation croisée 5-fold divise le jeu d'entraînement en 5 parties. Le modèle est entraîné 5 fois, chaque fois sur 4 parties et évalué sur la 5e. Le faible écart-type observé sur tous les modèles confirme leur stabilité et leur capacité à généraliser.

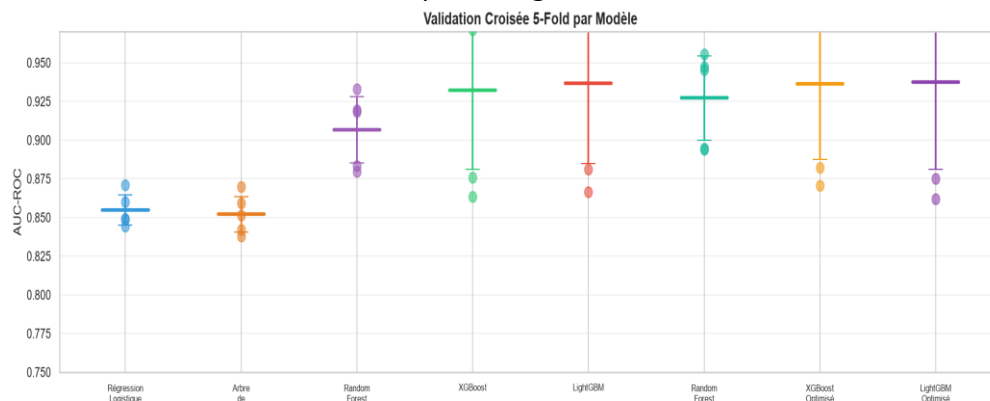


Figure 8. Validation croisée 5-fold - stabilité de l'AUC-ROC par modèle.

Seuil de décision optimisé

Par défaut, un modèle prédit "Churner" si la probabilité calculée dépasse 0.50. Ce seuil n'est pas optimal dans notre contexte : rater un churner (faux négatif) est bien plus coûteux que générer une fausse alerte (faux positif). Une recherche exhaustive sur 80 valeurs de seuil entre 0.10 et 0.90 a identifié 0.10 comme le seuil maximisant le F1-score.



Figure 9. Optimisation du seuil de décision dans le dashboard Qualité & Monitoring.

Courbe d'apprentissage

Les scores d'entraînement et de validation convergent avec un faible écart, confirmant l'absence de surapprentissage significatif sur XGBoost.

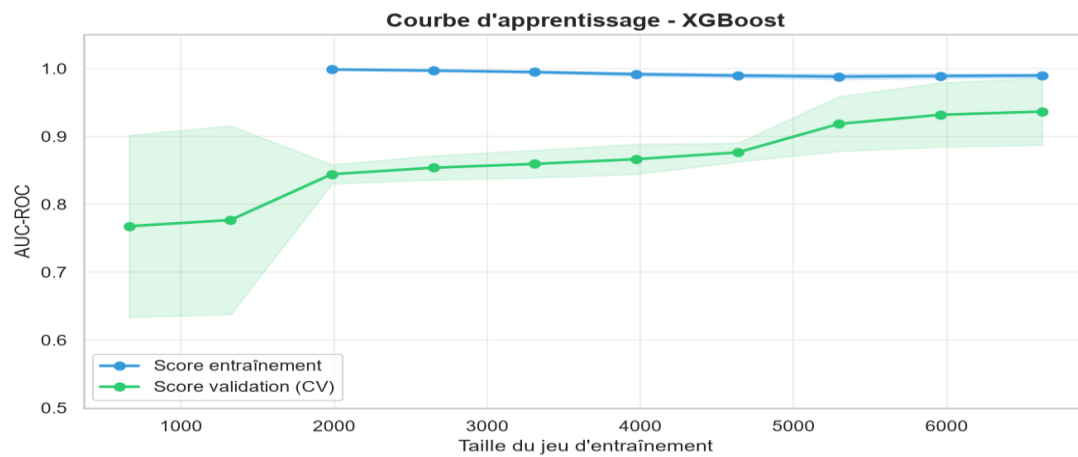


Figure 10. Courbe d'apprentissage - convergence des scores train et validation.

Résultats comparatifs

Modèle	Accuracy	Recall	F1	AUC-ROC
Régression Logistique	73.8 %	79.7 %	0.62	0.8405
Arbre de Décision	74.4 %	75.4 %	0.61	0.8186
Random Forest	77.6 %	65.5 %	0.61	0.8319
XGBoost *	75.0 %	75.7 %	0.62	0.8266
LightGBM	77.4 %	60.4 %	0.59	0.8284

* Modèle retenu. Sur 374 clients ayant réellement churné dans le jeu de test, le modèle en détecte 283 (75.7 %).

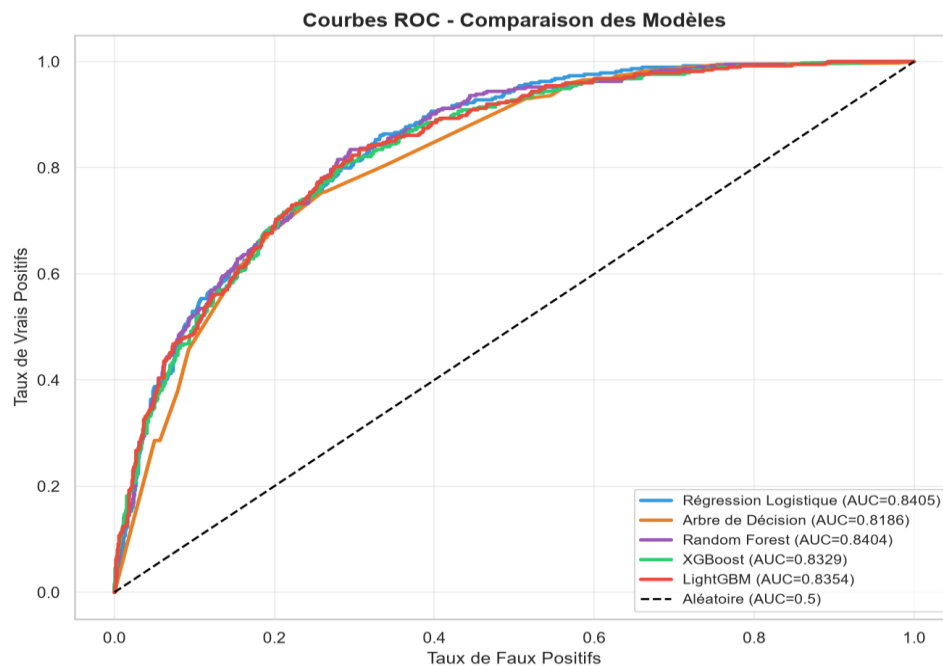


Figure 11. Courbes ROC comparatives des 8 modèles.

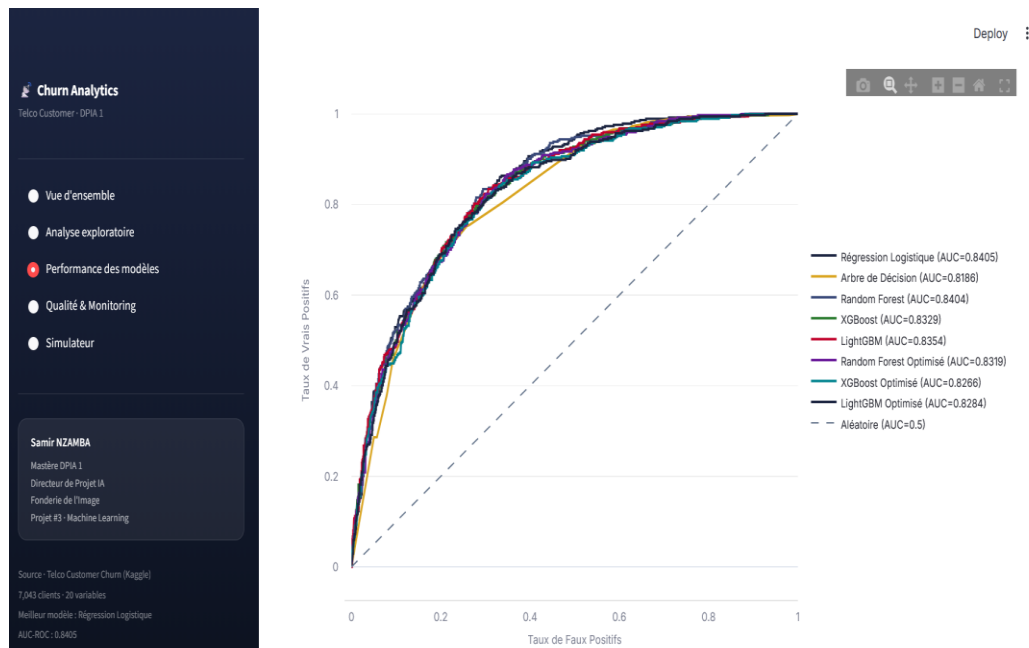


Figure 12. Courbes ROC interactives dans le dashboard.

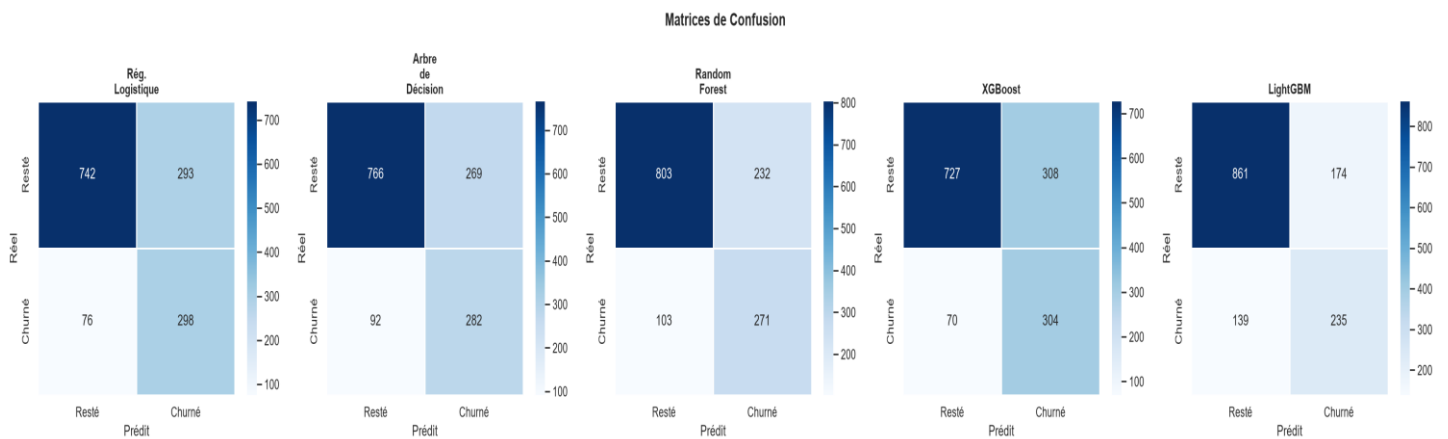


Figure 13. Matrices de confusion comparatives.

7) Feature importance et explicabilité SHAP

Deux niveaux d'interprétabilité ont été mis en place. La feature importance XGBoost mesure l'usage global de chaque variable dans les arbres. SHAP décompose la prédiction pour chaque client individuellement, répondant à la question : pourquoi le modèle a-t-il prédit ce résultat pour ce client précis ?

Feature importance globale

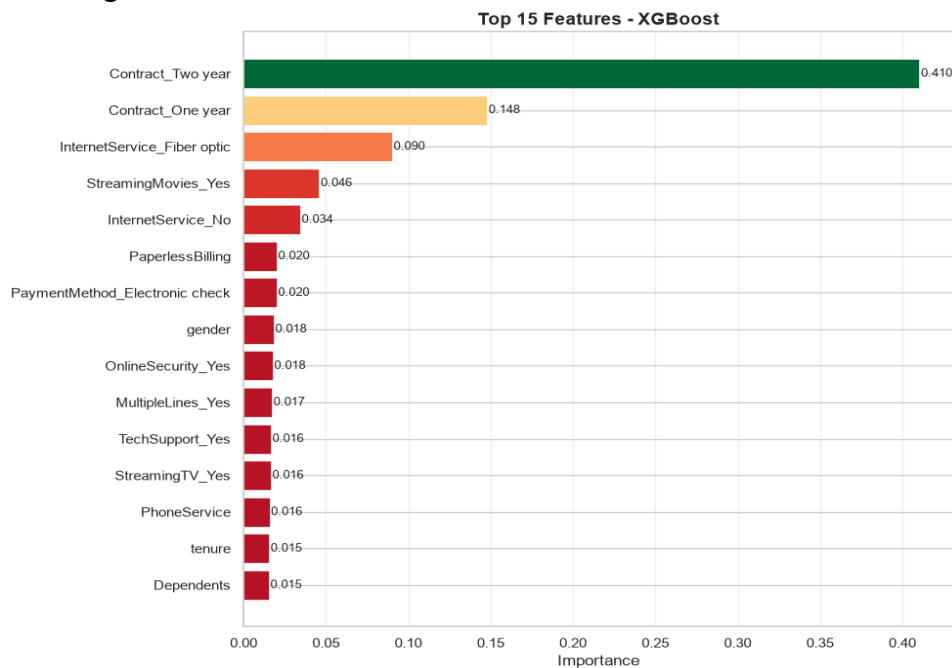


Figure 14. Top 15 features - XGBoost.

SHAP Summary Plot

Chaque point représente un client. Position droite = pousse vers le churn. Couleur rouge = valeur élevée de la variable. Le résultat notable : Contract_Two year est le facteur d'impact individuel le plus fort, devant tenure. Ce résultat affine la feature importance classique qui plaçait tenure en première position.

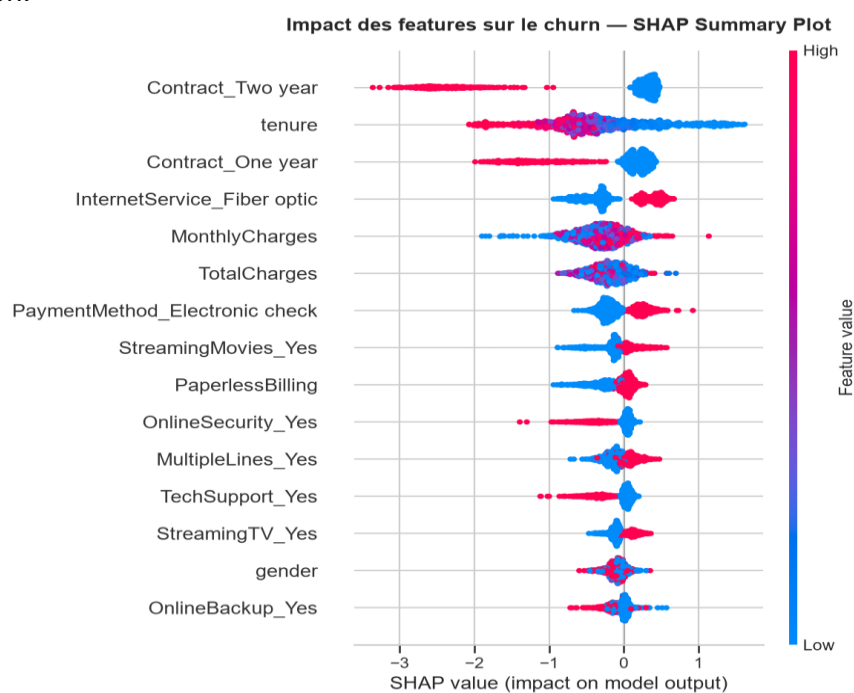


Figure 15. SHAP Summary Plot - impact de chaque variable sur le churn.

SHAP Waterfall Plot - explication individuelle

Pour un client présentant 78.8 % de risque, le graphique montre que le mode de paiement Electronic check (+0.37) et l'absence de contrat long (+0.35) contribuent le plus à l'élévation du risque. Ce niveau de détail permet de proposer une action de rétention ciblée : passage en contrat annuel ou modification du mode de paiement.

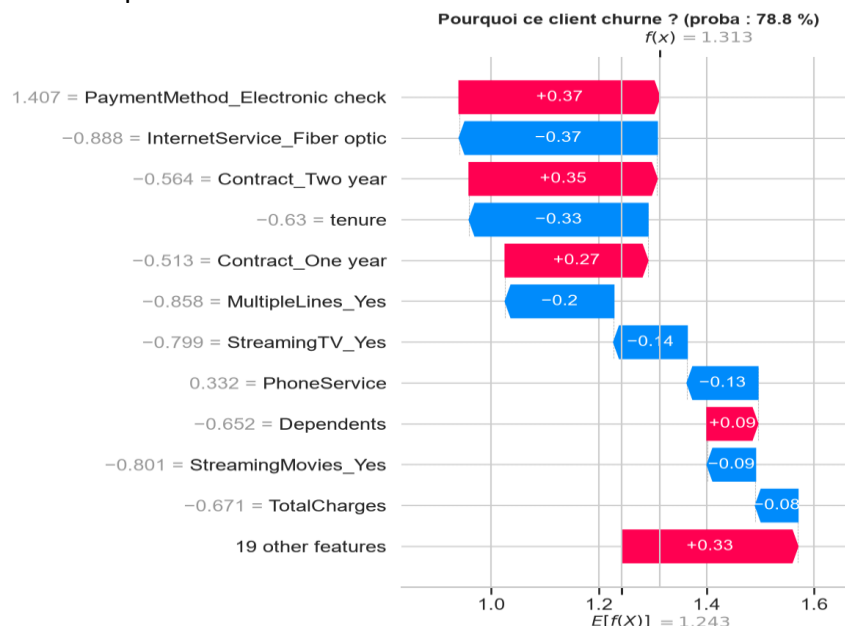


Figure 16. SHAP Waterfall Plot - explication d'une prédiction à 78.8% de risque.

SHAP Dependence Plot

Les clients récents avec mensualité élevée présentent le risque le plus fort. À mesure que l'ancienneté augmente, le risque diminue fortement et la mensualité perd de son pouvoir discriminant.

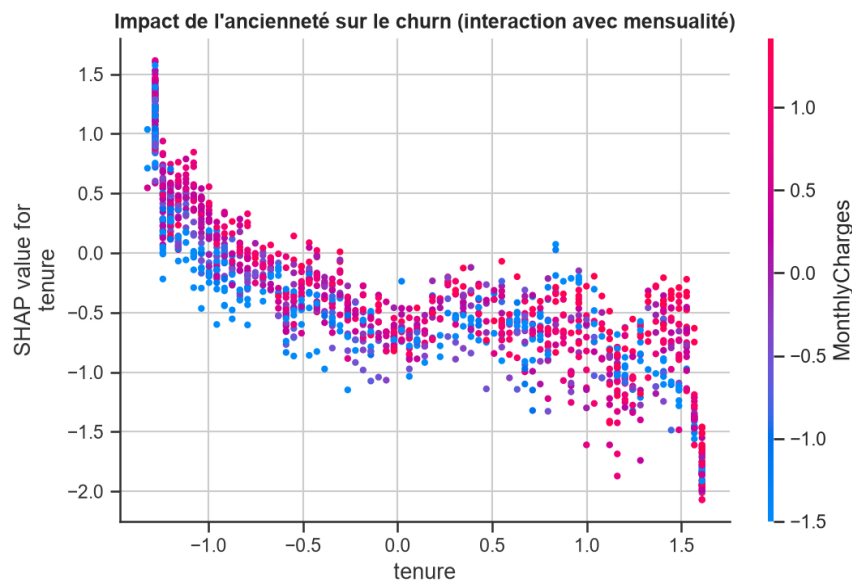


Figure 17. SHAP Dependence Plot - interaction ancienneté / mensualité.

Rang	Variable	SHAP moyen	Interprétation
1	Contract_Two year	0.813	Contrat long = protection massive contre le churn
2	tenure	0.656	Ancienneté élevée = risque réduit

Rang	Variable	SHAP moyen	Interprétation
3	Contract_One year	0.414	Engagement intermédiaire = protection partielle
4	InternetService_Fiber optic	0.392	Fiber optic = risque accru (paradoxe à investiguer)
5	MonthlyCharges	0.350	Mensualité élevée = risque accru

8) Dashboard et API REST

Dashboard Streamlit

Le dashboard est accessible en ligne sur <https://nzb-churn-project.streamlit.app/>. Il est développé avec Streamlit et Plotly, compatible light/dark mode, et organisé en cinq pages.



Figure 18. Page Vue d'ensemble du dashboard.

Vue d'ensemble. KPIs clés, résumé de la démarche, facteurs de risque majeurs.

Analyse exploratoire. Visualisations EDA interactives avec filtres par variable, histogrammes et boxplots.

Performance des modèles. Courbes ROC comparatives, matrices de confusion interactives, feature importance, validation croisée.

Qualité & Monitoring. Optimisation du seuil de décision, courbe d'apprentissage, rapport Evidently HTML intégré.

Simulateur. Prédiction temps réel pour un profil client avec jauge de probabilité, facteurs de risque détectés et recommandations. Un avertissement s'affiche si les valeurs saisies dépassent les bornes du dataset d'entraînement.



Figure 19. Page Simulateur

API REST FastAPI

Le modèle est exposé via une API REST développée avec FastAPI. Elle permet à n'importe quelle application externe d'interroger le modèle en envoyant un profil client au format JSON. La documentation interactive est générée automatiquement.

Churn Prediction API 1.0.0 OAS 3.1

/openapi.json

API de prédiction du churn client - Projet #3 DPIA 1

default

GET	/ Root	✓
GET	/health Health	✓
POST	/predict Predict	✓
POST	/predict/batch Predict Batch	✓
Schemas		

Figure 20. Documentation interactive FastAPI - 4 endpoints disponibles.

Méthode	Route	Description
GET	/	Statut de l'API
GET	/health	Informations du modèle en service
POST	/predict	Prédiction individuelle avec facteurs SHAP
POST	/predict/batch	Prédiction en lot (max 100 clients)

9) Génie logiciel et qualité du code

Tests unitaires

6 tests unitaires couvrent les fonctions critiques du module preprocessing.py. Ils garantissent que toute modification du code de nettoyage ne casse pas silencieusement le pipeline.

```
(venv)
samir@DESKTOP-PVEMP35 MINGW64 ~/Desktop/churn_project (main)
$ pytest tests/ -v
===== test session starts =====
platform win32 -- Python 3.13.5, pytest-9.1.0, pluggy-1.6.0 -- C:\Users\samir\Desktop\churn_project\venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\samir\Desktop\churn_project
plugins: anyio-4.14.0, Faker-40.23.0
collected 6 items

tests/test_preprocessing.py::test_clean_data_removes_customer_id PASSED [ 16%]
tests/test_preprocessing.py::test_clean_data_fixes_total_charges PASSED [ 33%]
tests/test_preprocessing.py::test_clean_data_handles_empty_total_charges PASSED [ 50%]
tests/test_preprocessing.py::test_clean_data_converts_senior_citizen PASSED [ 66%]
tests/test_preprocessing.py::test_encode_features_produces_numeric_columns PASSED [ 83%]
tests/test_preprocessing.py::test_encode_features_keeps_same_row_count PASSED [100%]

===== 6 passed in 2.73s =====
(venv)
samir@DESKTOP-PVEMP35 MINGW64 ~/Desktop/churn_project (main)
$
```

Figure 21. Résultat pytest - 6 tests passés en 2.73 secondes.

Test	Ce qu'il vérifie
test_clean_data_removes_customer_id	customerID est bien supprimé
test_clean_data_fixes_total_charges	TotalCharges est converti en float
test_clean_data_handles_empty_total_charges	Valeurs vides remplacées par 0
test_clean_data_converts_senior_citizen	Harmonisation 0/1 vers No/Yes
test_encode_features_produces_numeric_columns	Aucune colonne texte après encodage
test_encode_features_keeps_same_row_count	Aucune ligne perdue à l'encodage

Pipeline CI/CD GitHub Actions

Un pipeline GitHub Actions se déclenche automatiquement à chaque push sur la branche principale. Il garantit qu'aucune modification ne casse silencieusement le pipeline.

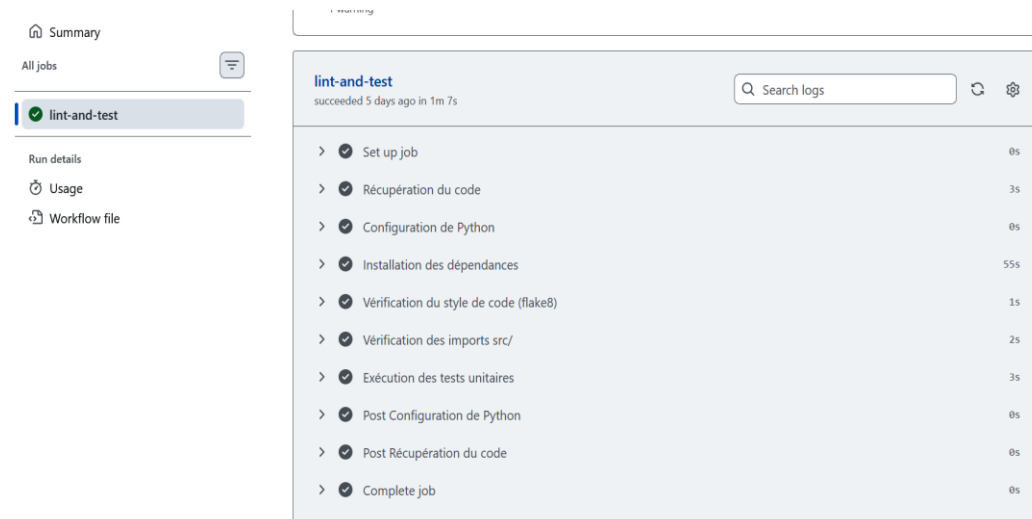


Figure 22. Pipeline CI/CD GitHub Actions - toutes les étapes au vert.

Le pipeline comprend 7 étapes : récupération du code, configuration de Python 3.11, installation des dépendances, vérification du style de code avec flake8 (conformité PEP 8), vérification des imports des modules src/, exécution des tests unitaires, et finalisation du job.

Monitoring Evidently

Evidently compare les données d'entraînement (référence) aux données de test (courant) sur deux dimensions : performance de classification et data drift. Le rapport HTML est intégré directement dans le dashboard (page Qualité & Monitoring). En production réelle, il serait régénéré à intervalles réguliers et déclencherait un réentraînement du modèle en cas de drift détecté au-delà d'un seuil prédéfini.

Outils utilisés pour le calcul numérique

Librairie	Rôle	Version
NumPy	Calcul numérique vectorisé	2.4.6
pandas	Manipulation de DataFrames	3.0.3
scikit-learn	Modèles ML, métriques, preprocessing	1.9.0
XGBoost	Gradient Boosting	3.2.0
LightGBM	Gradient Boosting rapide	4.6.0
imbalanced-learn	SMOTE	0.14.2
SHAP	Explicabilité des modèles	0.52.0
Evidently	Monitoring data drift	0.7.21

10) Recommandations métier

Les résultats du modèle et l'analyse SHAP permettent de formuler trois recommandations concrètes à destination des équipes commerciales.

Priorité 1 - Cibler les nouveaux clients dans les 12 premiers mois

Les clients ayant moins de 12 mois d'ancienneté churent à 47.7 %, soit près d'un sur deux. Il est recommandé de mettre en place un programme de bienvenue renforcé avec des points de contact proactifs à J+30, J+60 et J+90, ainsi qu'une offre de fidélisation dès le troisième mois.

Priorité 2 - Inciter au passage en contrat long

C'est le levier le plus puissant identifié par SHAP. Les clients en contrat deux ans churent à 2.8 % contre 42.7 % pour les clients en contrat mensuel. Des réductions significatives ou des avantages substantiels pour les clients qui s'engagent sur un ou deux ans seraient directement rentables.

Priorité 3 - Investiguer le service Fiber optic

Le taux de churn de 41.9 % sur ce service premium est paradoxal. Ce signal peut indiquer un problème de qualité réseau ou un rapport qualité-prix insuffisant face à la concurrence. Une analyse de satisfaction spécifique sur ce segment est recommandée.

11) Gestion de projet et GitHub

Organisation

Le projet a été construit en quatre phases successives : mise en place de l'environnement et du dépôt GitHub, exploration et préparation des données (notebook 01), modélisation, optimisation, SHAP et monitoring (notebook 02), développement du dashboard, de l'API et déploiement.

Dépôt GitHub

Disponible sur https://github.com/SNZAMBA65/churn_project. Commits conventionnels en français avec un type préfixé (feat:, fix:, docs:, mise à jour:). Les modèles sérialisés et les exports sont versionnés pour permettre le déploiement sur Streamlit Cloud sans réentraînement.

12) Conclusion et perspectives

Bilan

Les objectifs du projet sont atteints. Le pipeline fonctionne de bout en bout depuis le chargement des données brutes jusqu'au dashboard déployé en ligne. XGBoost atteint un AUC-ROC de 0.8266 et détecte 75.7 % des churners réels sur le jeu de test, avec une stabilité confirmée par la validation croisée 5-fold.

Sur le plan technique, le projet couvre l'ensemble de la chaîne data et ML moderne : nettoyage, encodage, SMOTE, comparaison multi-algorithmes, GridSearchCV, SHAP, Evidently, Streamlit Cloud, FastAPI, tests unitaires et CI/CD.

Limites

Trois limites principales doivent être mentionnées. Le modèle est entraîné sur un dataset spécifique à un opérateur télécom américain, ses performances ne sont pas garanties sur d'autres marchés. Les données sont statiques et le modèle ne se met pas à jour automatiquement. Enfin, la précision reste modeste (~54 %), compromis intentionnel justifié par le contexte métier mais à communiquer clairement aux équipes utilisatrices.

Perspectives

Trois perspectives ont été identifiées. L'intégration de SHAP directement dans le simulateur pour accompagner chaque prédiction d'une explication individuelle visible. Le déploiement de l'API FastAPI sur un service cloud pour la rendre accessible en dehors de l'environnement local. La mise en place d'un réentraînement automatique déclenché par Evidently lorsque le data drift dépasse un seuil prédéfini, ce qui rendrait le système vraiment autonome en production.

Annexes**Stack technique complète**

Python 3.13, pandas 3.0.3, numpy 2.4.6, matplotlib 3.11.0, seaborn 0.13.2, scikit-learn 1.9.0, xgboost 3.2.0, lightgbm 4.6.0, imbalanced-learn 0.14.2, shap 0.52.0, plotly 5.24.1, streamlit 1.58.0, fastapi 0.141.1, uvicorn 0.49.0, evidently 0.7.21, joblib 1.5.3, pytest 9.1.0, flake8 7.3.0, GitHub Actions.

Liens

Dashboard en ligne : <https://nzb-churn-project.streamlit.app/>

Dépôt GitHub : https://github.com/SNZAMBA65/churn_project

API REST (locale) : <http://localhost:8000/docs>

Dataset source : <https://www.kaggle.com/datasets/blastchar/telco-customer-churn>